ArtsibEro:

An Intuitive and Accessible Document Management System for a Modern School System

MaPaSa

Fiona Macapulay

Katrin Paraguya

Brandon Sayo

General Roxas Elementary School

https://cs191-mapasa.github.io/blog/

CS 191

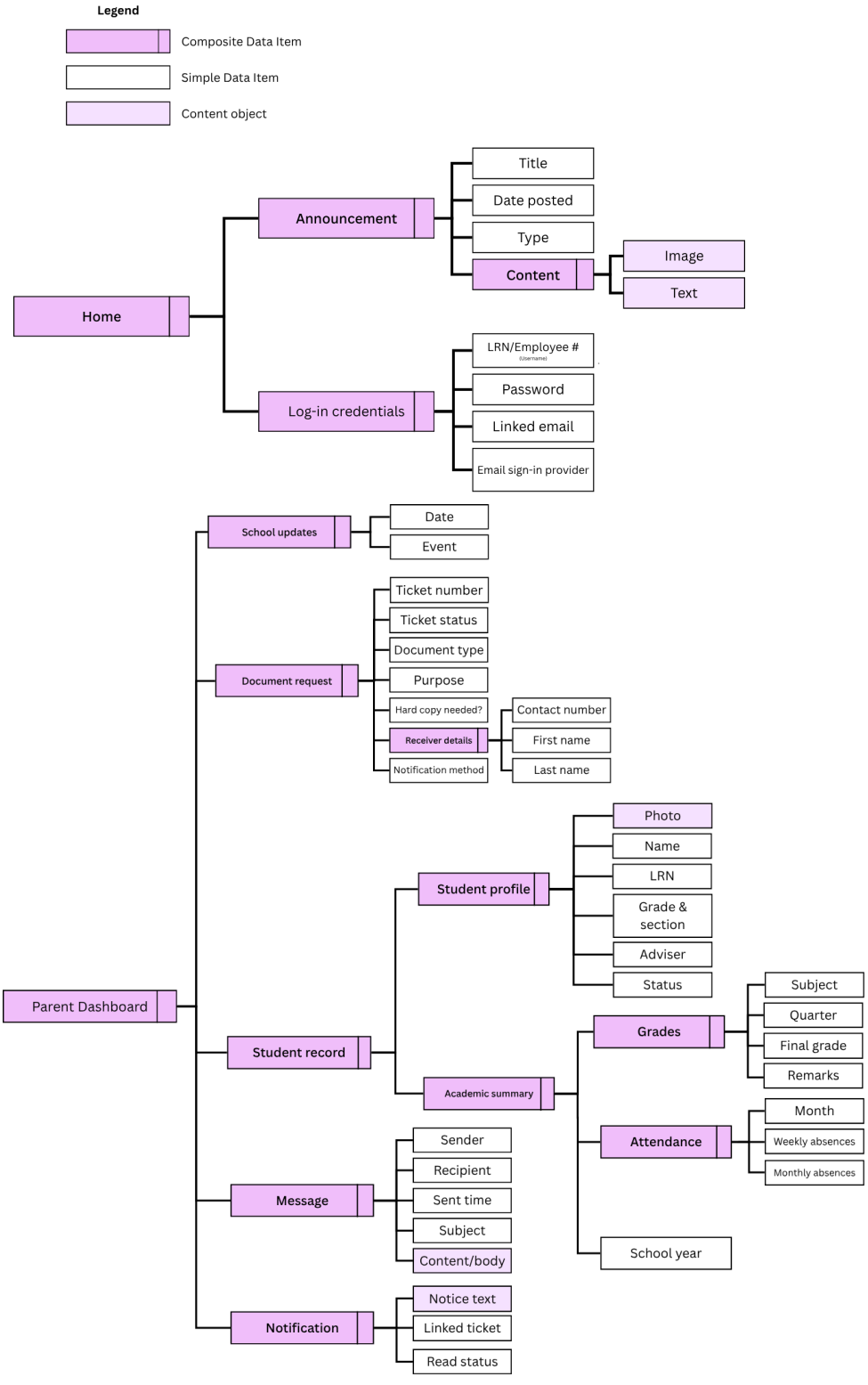
WFQ1

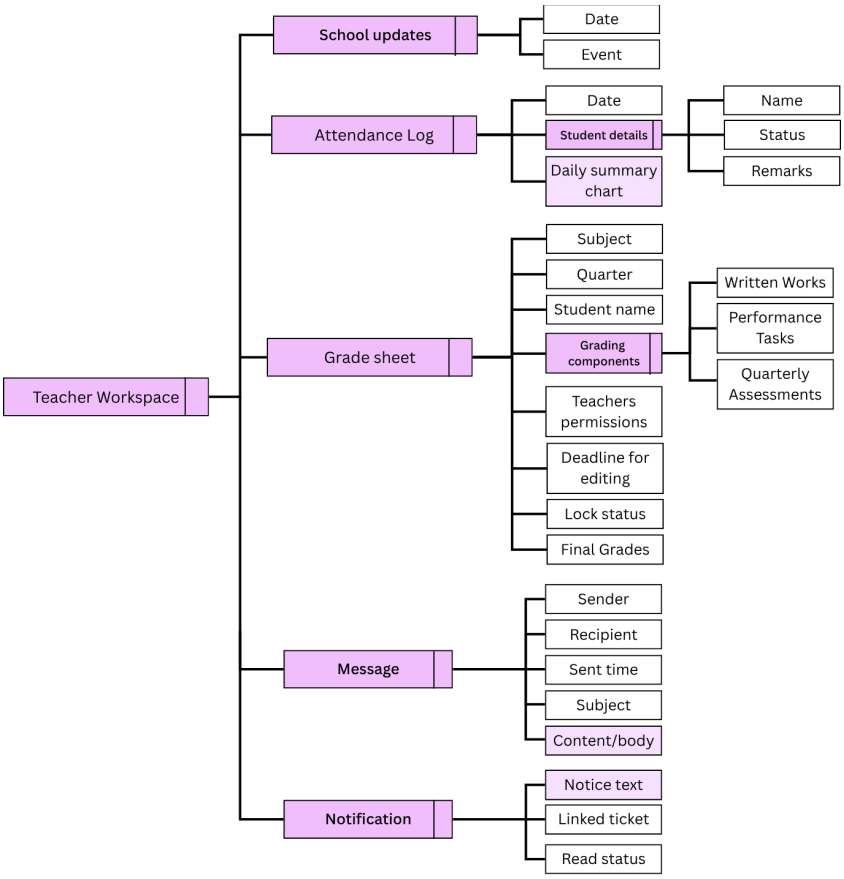
Ma'am Figueroa

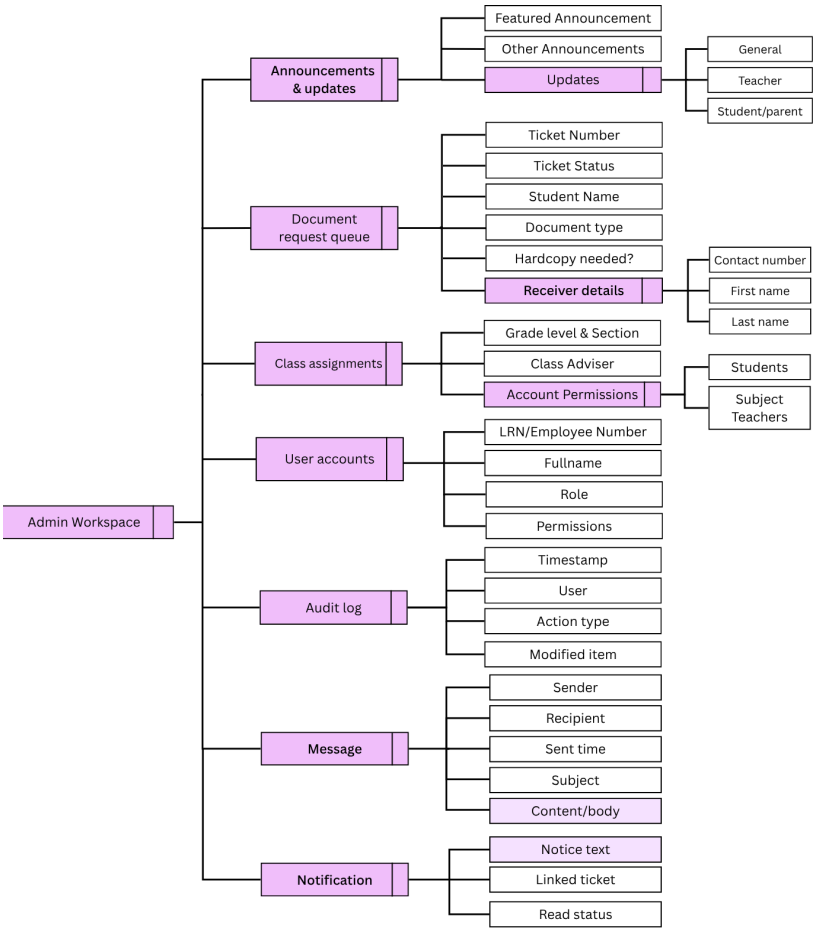
System Requirements Document

I. Web App Requirements Model

a. Content Model




Teacher Workspace
School updates
Date
Event
Attendance Log
Date
Student details
Name
Status
Remarks
Daily summary chart
Grade sheet
Subject
Quarter
Student name
Grading components
Written Works
Performance Tasks
Quarterly Assessments
Teachers permissions
Deadline for editing
Lock status
Final Grades
Message
Sender
Recipient
Sent time
Subject
Content/body
Notification
Notice text
Linked ticket
Read status


Admin Workspace
Announcements & updates
Featured Announcement
Other Announcements
Updates
General
Teacher
Student/parent
Document request queue
Ticket Number
Ticket Status
Student Name
Document type
Hardcopy needed?
Receiver details
Contact number
First name
Last name
Class assignments
Grade level & Section
Class Adviser
Account Permissions
Students
Subject Teachers
User accounts
LRN/Employee Number
Fullname
Role
Permissions
Audit log
Timestamp
User
Action type
Modified item
Message
Sender
Recipient
Sent time
Subject
Content/body
Notification
Notice text
Linked ticket
Read status

b. Interaction Model

i. Actors and Goals

Types	*Actors*	*Goals*
Initiating	Guardian	- log into portal - view announcements - view student's documents - create document ticket - track document ticket - send messages
	Teacher	- log into portal - view announcements - manage student's documents - process document ticket - send messages
	Administrator	- log into portal - manage announcements - process document ticket - send messages
Participating	System	- manage initiating actors' actions

ii. Use Cases

1. *Account verification*

   - Guardians can verify their email if it has been encoded into the system.

2. *Account setup*

   - Employees can set-up their email for the system.

3. *Manage accounts*

   - Administrators can manage accounts currently in the system.

4. *Manage document*

   - Users can view personal documents and encode new ones.

5. *Create request ticket*

   - Guardians can request a document.

6. *Process request ticket*

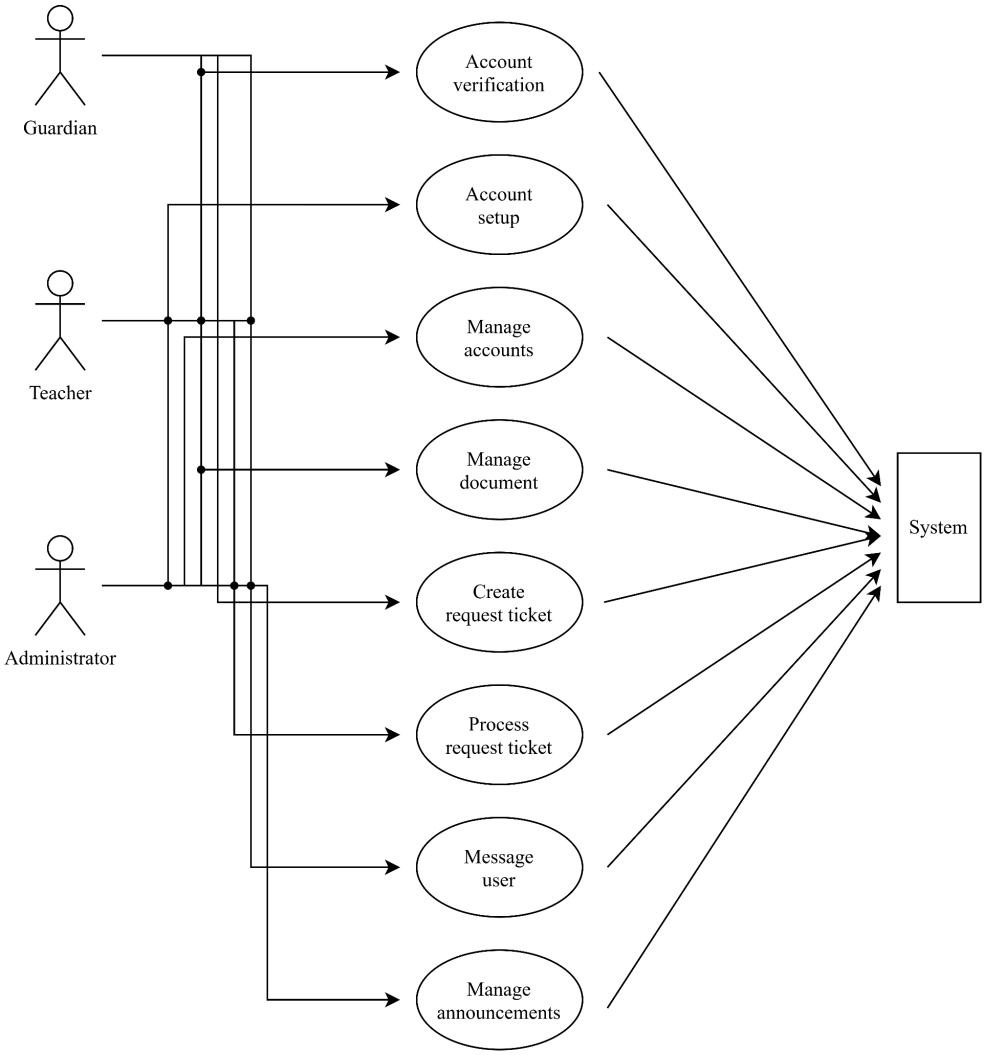
   - Relevant employees can process a document request.

7. *Message user*

   - Users can privately message other users.

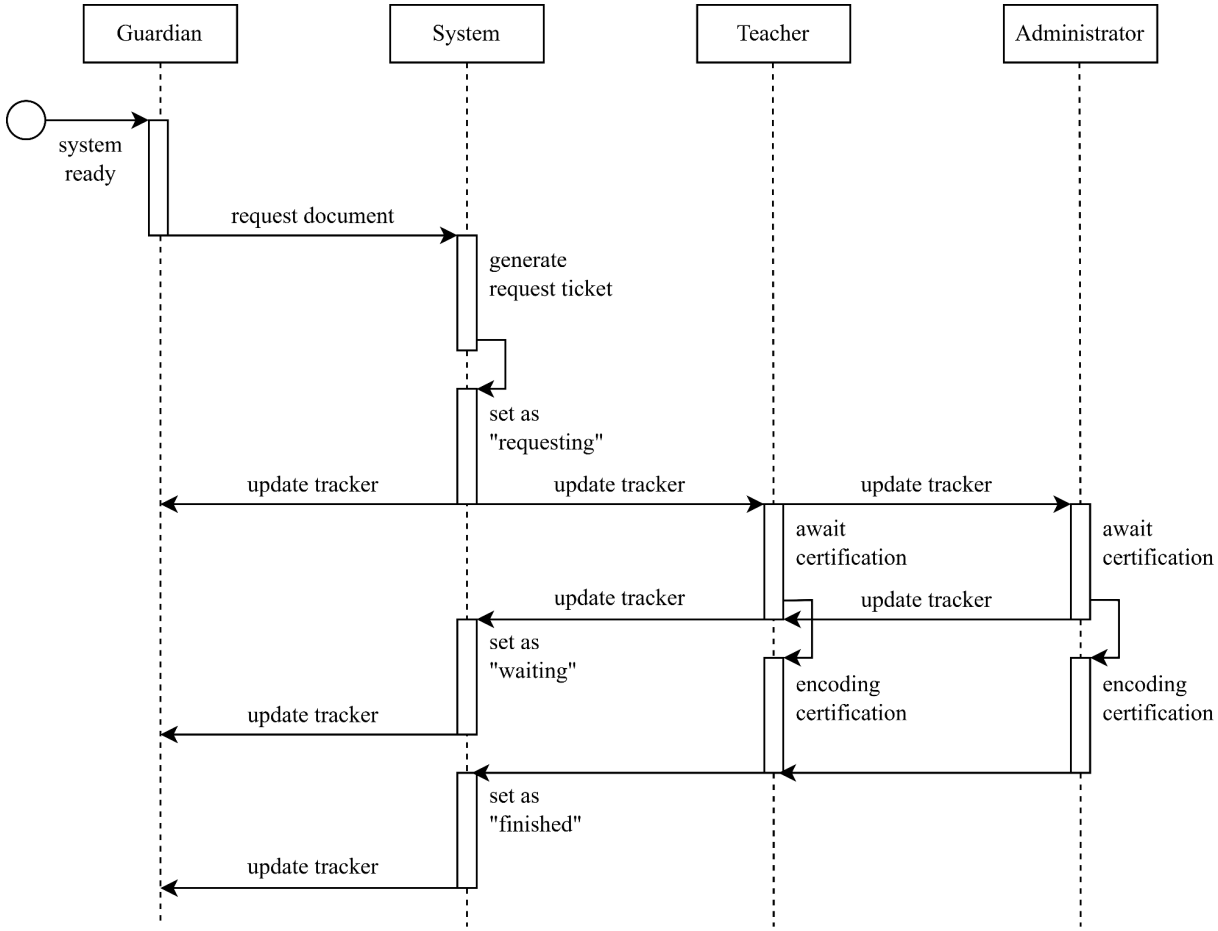
8. *Manage announcements*

   - Administrators can manage the current announcements.

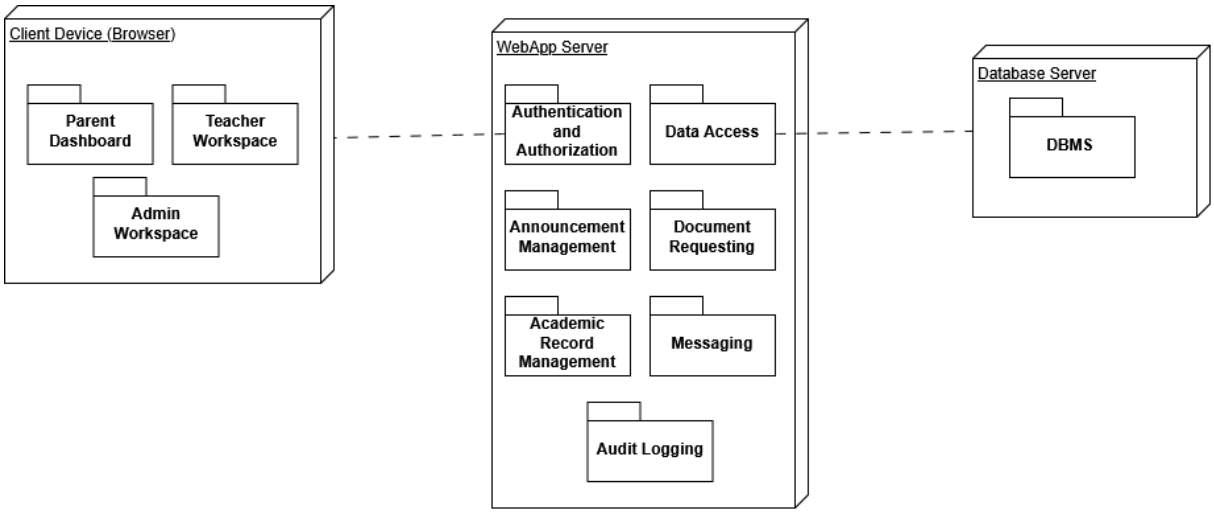


iii. System Sequence Diagrams

Create & Process request ticket



c. Configuration Model



II. Web App Architecture and Design

a. Architecture Design

Architectural Styles

- *Architecture Genre:* Communications

  Arsibero aims to bridge communication gaps present within the current school system's document processing. It will facilitate messaging and streamline requesting especially between guardians and the school's teachers or administrators.

- *Content Architecture*: Linear with diversion structure

  Since the aim is an intuitive role-based system, a linear-with-diversion content structure would allow for step-by-step workflows. Diversions represent domains of operations which themselves give way to specific contextual operations.

- *WebApp Architecture*: Model-View-Controller (MVC)

  Following an object-oriented structure, the system is split into a model, view and controller.

a. Model:

   This is the aspect of the system that manages the core logic with respect to the relevant user roles and objects such as documents or request tickets manipulated by the users within the system.

b. View:

   This is the aspect of the system responsible for the graphical user interface, consistent between users but also facilitating each possible role's capabilities and goals.

c. Controller:

   This is the aspect of the system that communicates and validates the inputs of the users given their role to the core system model and then commuting its results back to the user.

Quality Requirement Tree

To ensure that Artisibero is up to quality, we describe how it addresses certain relevant attributes as described by Pressman's (2010) quality requirement tree:

1. *Usability*

   Global site and interface understandability would be maintained through consistent and clear interface design with capabilities for help features that guide and lead users to their inferred goal. Accessibility and art passes will also be done to ensure a pleasing and inclusive experience. Online feedback for messages, document releases, and document request updates will be facilitated through notification alerts, as well as a bulletin board for site-wide announcements.

2. *Functionality*

   The site will be built on top of a database management system to allow for efficient, reliable, and secure searching and retrieving capabilities for user document management. As mentioned before, the use of a linear content structure supported by MVC architecture will help create a comfortable and intuitive navigation and browsing experience for each user role.

3. *Reliability*

   Standard quality coding practices will be used, such as SOLID, to implement the MVC structure of the system to ensure robust class correspondence, correct link processing, and user input validation. In addition to this, logs will be implemented to ensure easy error recovery and debugging.

4. *Availability and Efficiency*

   The application would be designed and tested on a modern browser with low hardware specifications to ensure that all users can use the system reliably. And as a relatively feature light web application requiring minimal client-side resources, following good coding practices will allow for responsive performance and efficient generation.

5. *Scalability and Maintainability*

   As an MVC structure built on object-oriented structures, the system will be very modular that cleanly separates user interface components from logical classes that would allow for ease of correction, adaptability, and extensibility.

6. *Security*

   Standard authentication practices to ensure privacy and a role-based access control list will be used to separate the guardian, teacher, and administrator permissions.

Architectural Decisions

Finally, we enumerate the core architectural decisions made for Artsibero.

1. *Server-based Website Application*

   At its core, this project aims to allow ease of communication between guardians and the relevant school personnel. Implementing the system as an online site allows for a great degree of freedom in allowing any guardian or school staff to access the site, to some extent, from anywhere as long as they have a device that can connect to Artsibero. An

application requiring installation or any sort of extended setup is far too heavy handed and very cumbersome relative to the goals of the users involved.

2. *MVC Architectural Pattern*

    As mentioned so far, this object-oriented architecture allows for proper separation of concerns, allowing a far greater magnitude of modularity which in turn allows for a lot of flexibility, extensibility, and testability. A monolithic structure will be far too hard to maintain and debug.

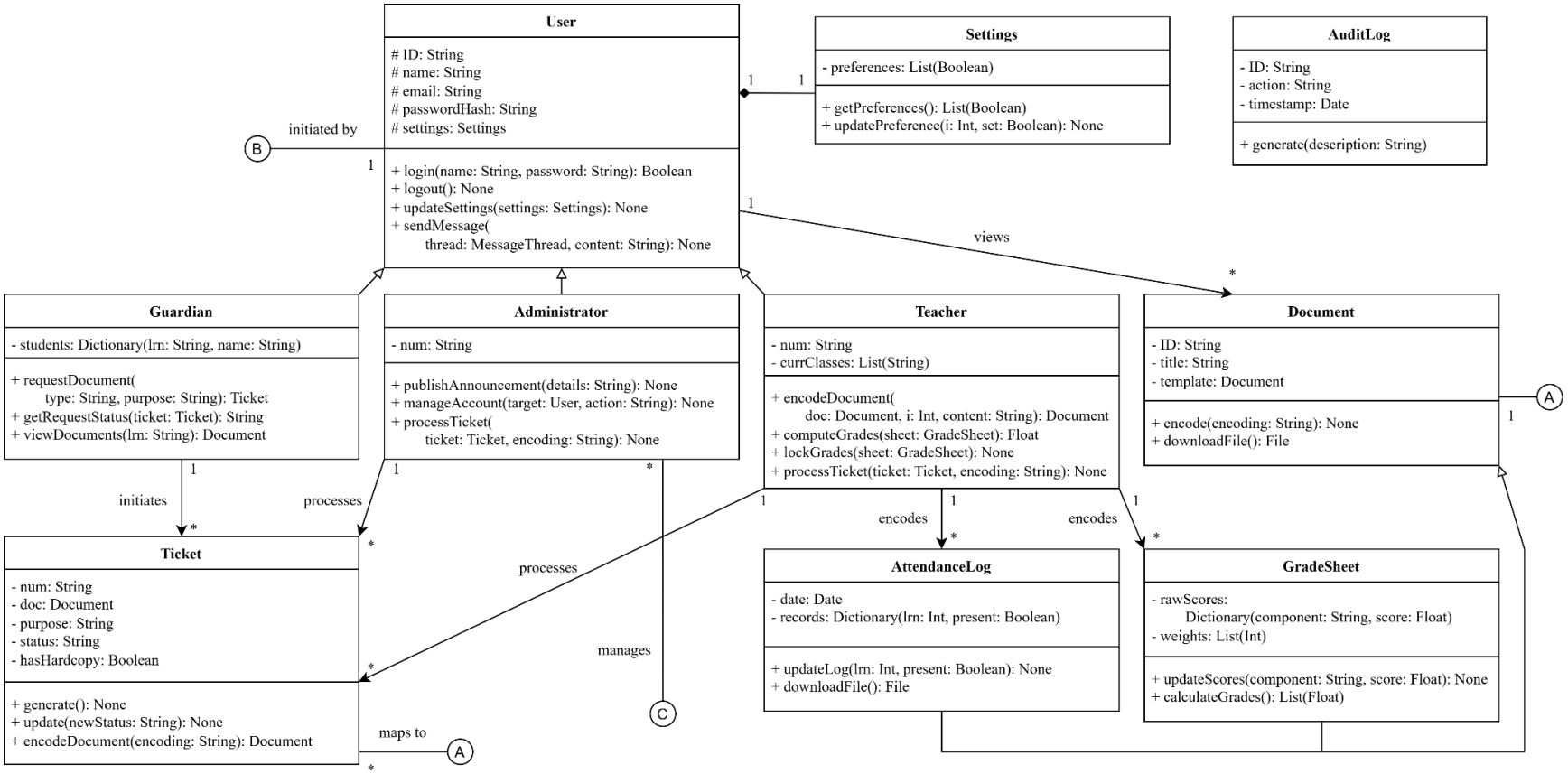
3. *Role-Based Access Control*

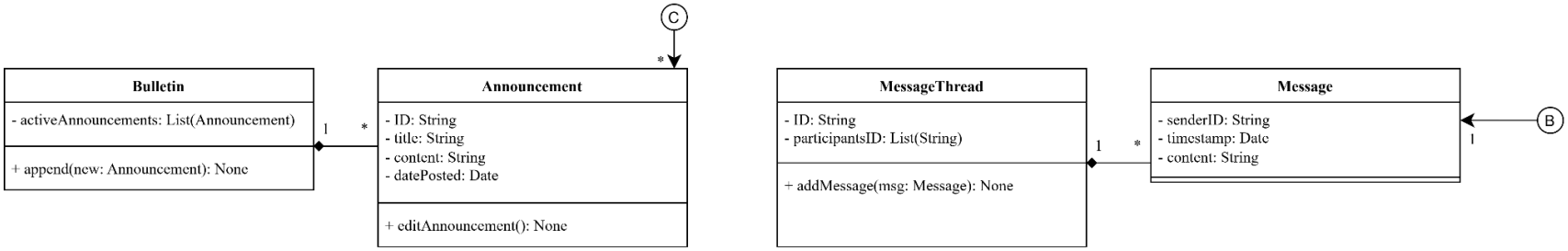
    Since this system is heavily centered on the three possible roles interacting with the system that manages a collection of documents that aren't tied to the permissions set by its "owner", a role-based access control system is best middle-ground for effectivity and simplicity. Other more attributes-based (IMI, 2020) or capabilities-based systems are too complicated and misaligned with the type of database managed by the system.

b. Content Design

i. Class Diagram

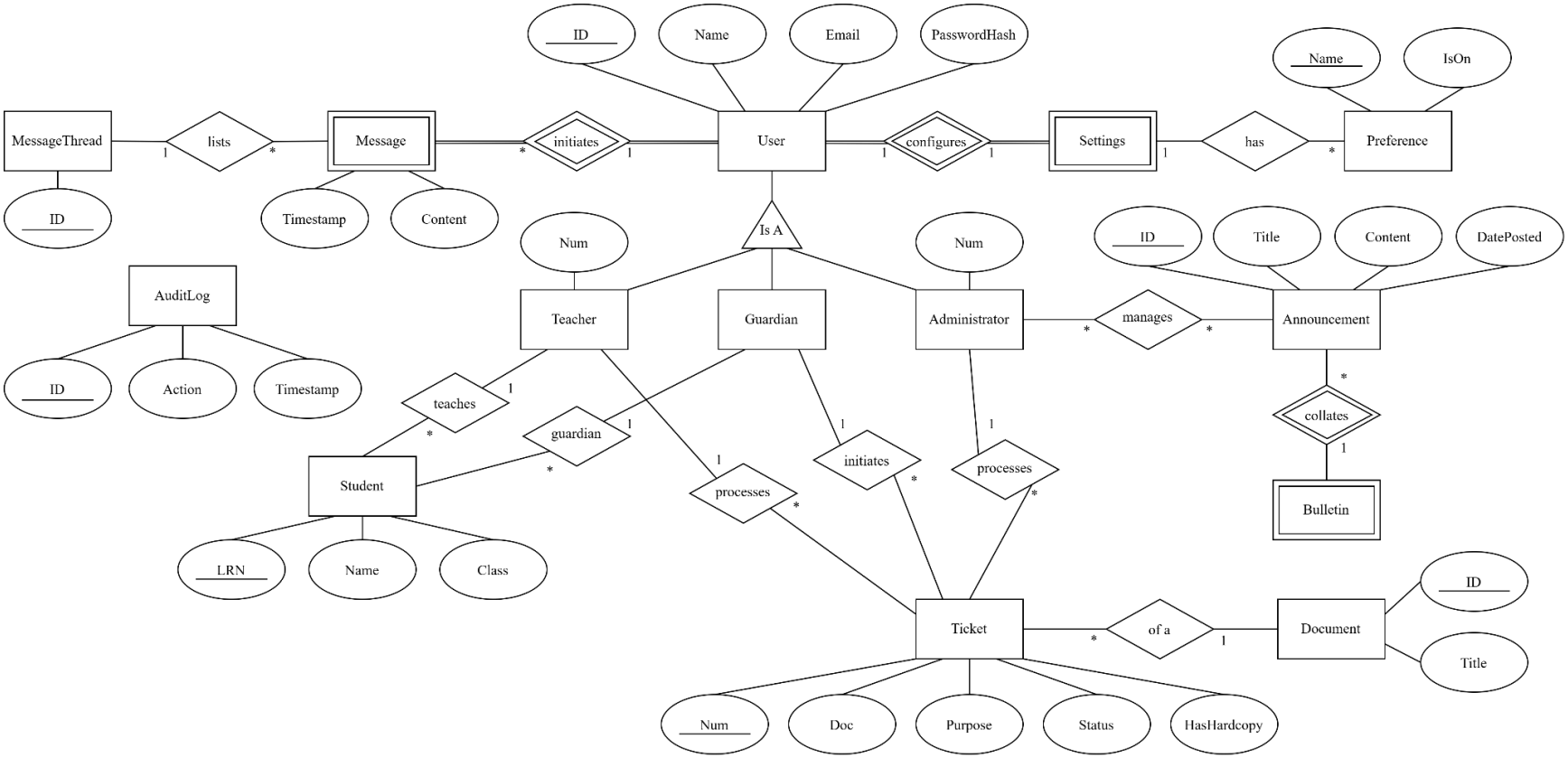
The Unified Modeling Language diagram for the system is shown below.




C
*
Bulletin
- activeAnnouncements: List(Announcement)
+ append(new: Announcement): None
1
*
Announcement
- ID: String
- title: String
- content: String
- datePosted: Date
+ editAnnouncement(): None
MessageThread
- ID: String
- participantsID: List(String)
+ addMessage(msg: Message): None
1
*
Message
- senderID: String
- timestamp: Date
- content: String
1
B

ii. Entity-Relationship Diagram

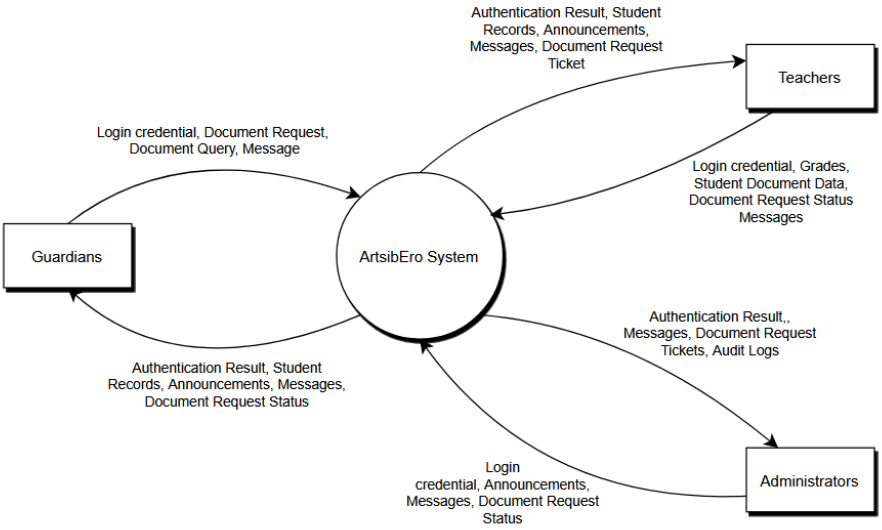
The entity-relationship diagram of the system in Chen notation is shown below.



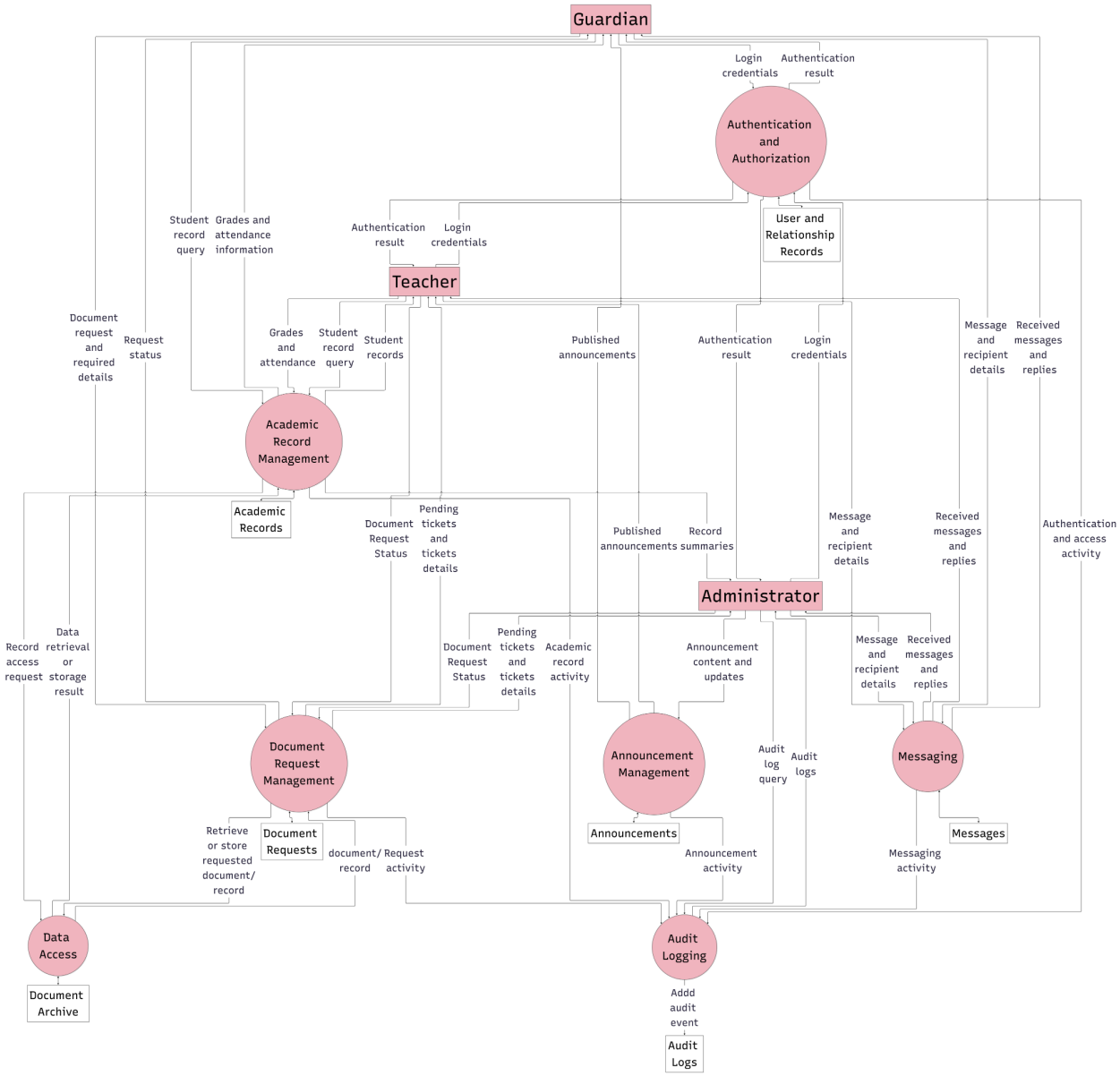
iii. Data Flow Diagrams

The following are the data-flow diagrams of the system:

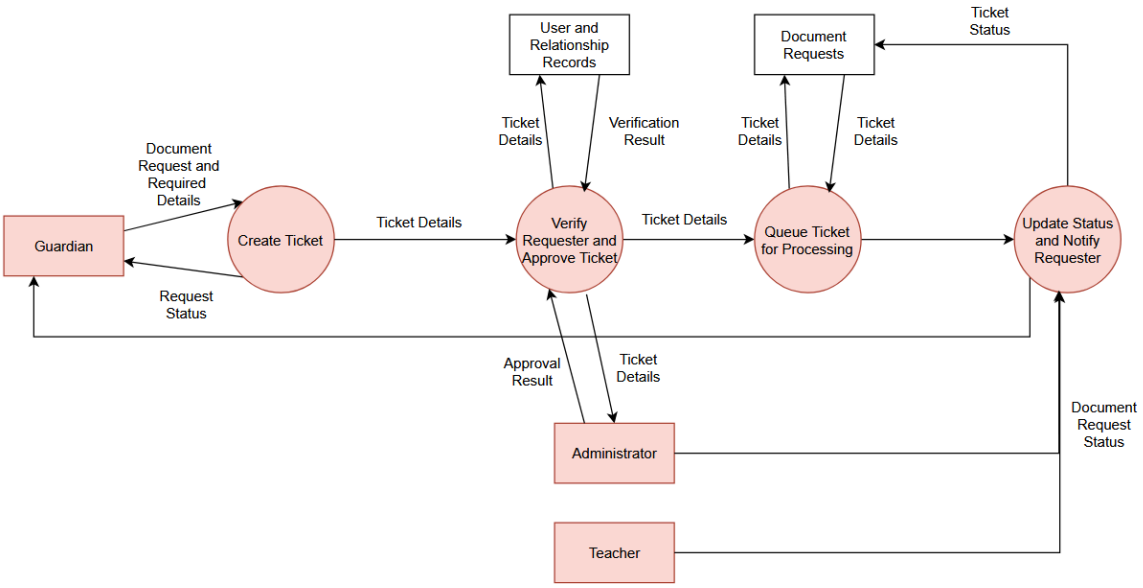
DFD Level 0



DFD Level 1


Guardian
Login credentials
Authentication result
Authentication and Authorization
User and Relationship Records
Student record query
Grades and attendance information
Authentication result
Login credentials
Teacher
Document request and required details
Request status
Grades and attendance
Student record query
Student records
Published announcements
Authentication result
Login credentials
Message and recipient details
Received messages and replies
Academic Record Management
Academic Records
Document Request Status
Pending tickets and tickets details
Published announcements
Record summaries
Message and recipient details
Received messages and replies
Authentication and access activity
Administrator
Record access request
Data retrieval or storage result
Document Request Status
Pending tickets and tickets details
Academic record activity
Announcement content and updates
Message and recipient details
Received messages and replies
Document Request Management
Announcement Management
Messaging
Audit log query
Audit logs
Retrieve or store requested document/ record
Document Requests
document/ record
Request activity
Announcements
Announcement activity
Messaging activity
Messages
Data Access
Audit Logging
Document Archive
Addd audit event
Audit Logs

DFD Level 2 for Document Requesting



c. Component-level Design

The system has seven major components: **Authentication and Authorization, Academic Record Management, Announcement Management, Document Requesting, Messaging, Audit Logging and Data Access.** Each component has its own procedural processing steps, with Document Requesting having the most complex algorithm. The data structures to be used include queues for ticket management, arrays or lists for collections of records or objects, and objects/records for representing data such as users, records, messages, and document tickets. These structures were generally chosen based on their suitability for the data being handled, efficient retrieval and processing and for maintaining consistency across the system. The Commercial Off-The-Shelf (COTS) items may include an identity management system, a web server for handling off-campus requests, a web framework for application development, document-generation libraries for generating and formatting documents, and a database management system.

The following table is an overview of the components:

Component	Local Data Objects	Processing	Interface	Depends on
Authentication and Authorization	User, Session, RolePermissions	Verify credentials, create session, check role permission per request	validateSession(), checkPermission()	Data Access, Audit Logging
Academic Record Management	Grade, Attendance	Teachers upload and edit records, system formats and computes records, parents view them	uploadGrades(), recordAttendance( ), getRecords()	Authentication, Data Access, Audit Logging
Announcement Management	Announcement	Admin or teacher posts,, users view	postAnnouncemen t(), getAnnouncement s()	Authentication, Data Access
Document Requesting	Ticket	Submit, verify, approve or reject, generate the document, update status	submitRequest(), checkStatus(), updateRequest()	Authentication, Data Access, Audit Logging, Messaging
Messaging	Message	Send, store, mark as read	sendMessage(), listMessages()	Authentication, Data Access
Audit Logging	LogEntry	Receive events from other components and append them	recordEvent(), searchLog()	Data Access
Data Access	Query and result objects	Translate component calls into SQL and return results	query(), insert(), update()	DBMS

Authentication and Authorization

- *Local Data Objects*

        - **User**. One account per student's guardian, teacher or administrator.
        - **Session.** An active login.
        - **RolePermissions.** What each role may do.
- *Processing*
    1. Receive an action request with a session token and user information
    2. Look up permissions for the user's role
    3. Allow or deny the requested action
    4. Send event to Audit Logging
- *Interface*
    - validateSession(), checkPermission()
- *Dependencies*
    - Authentication and Authorization depends on Data Access to see role permissions and Audit Logging to write to the shared log. Every other component depends on it, making Authentication and Authorization a constrained dependency.

Academic Record Management

- *Local Data Objects*
    - **Grade.** Student's grade in a subject or assignment
    - **Attendance.** Student's attendance status per day
- *Processing*
    1. Teacher submits or edits grades or attendance for a class
    2. Check the teacher's permission through Authentication
    3. Validate and format each entry
    4. Compute necessary summaries (grade averages, attendance totals)
    5. Store the entries through Data Access
    6. Send event to Audit Logging
- *Interface*
    - uploadGrades(), recordAttendance(), getRecords()
- *Dependencies*

- Academic Record Management depends on Authentication to check permissions (constrained), Data Access to store and load records, and Audit Logging to write to the shared log.

Announcement Management

- *Local Data Objects*
  - **Announcement.** One announcement with a title, body and date posted
- *Processing*
  1. Admin or teacher submits an announcement
  2. Check posting permission through Authentication
  3. Store the announcement through Data Access
  4. When a user opens their dashboard, load recent announcements for their role and section
  5. Display them newest first
- *Interface*
  - postAnnouncement(), getAnnouncements()
- *Dependencies*
  - Announcement Management depends on Authentication to check permissions (constrained) and Data Access to store and load announcements.

Document Requesting

- *Local Data Objects*
  - **Ticket.** One document request with the requester, document type, status, timestamps, and remarks
- *Processing*
  1. Parent submits a ticket with a document type and purpose
  2. Admin verifies student details, request type and purpose
  3. If eligible, add the ticket to the queue. Otherwise show a rejection notice
  4. Administrator or Teacher updates the ticket status as it is processed
  5. Update the ticket status and notify the parent through Messaging
  6. Send event to Audit Logging

- *Interface*
    - submitRequest(), checkStatus(), updateRequest()
- *Dependencies*
    - Document Requesting depends on Authentication to check permissions (constrained), Data Access to store tickets, Audit Logging to write to the shared log, and Messaging to notify parents.

Messaging

- *Local Data Objects*
    - **Message.** One message between two users with the sender, recipient, body, and time sent.
- *Processing*
    1. User sends a message to a recipient
    2. Check that the sender may message that recipient through Authentication
    3. Store the message through Data Access
    4. When the recipient opens their inbox, load their messages newest first
- *Interface*
    - sendMessage(), listMessages()
- *Dependencies*
    - Messaging depends on Authentication to check permissions (constrained) and Data Access to store and load messages. Document Requesting depends on it for notifications.

Audit Logging

- *Local Data Objects*
    - **LogEntry.** One recorded event with the user, action, target, and time.
- *Processing*
    1. Receive an event from another component
    2. Stamp the event with the current date and time
    3. Append the entry to the log table in the database through Data Access
- *Interface*

    - recordEvent(), searchLog()
- *Dependencies*
    - Audit Logging depends on Data Access to store entries. Authentication, Academic Record Management, and Document Requesting write to it, making the log a shared dependency.

Data Access

- *Local Data Objects*
    - **Query.** One request to the database built from a component's call.
    - **Result.** The rows the database returns.
- *Processing*
    1. Receive a call from a component
    2. Build a SQL statement from it
    3. Send the statement to the DBMS
    4. Map the returned rows into objects
    5. Return the result, or an error if the statement failed
- *Interface*
    - query(), insert(), update()
- *Dependencies*
    - Data Access depends on the DBMS to run its statements. Every component that stores or reads data depends on it.

COTS

1. *Identity Management System*

   An existing identity management solution will be considered to securely handle user authentication and identity verification. Since the system processes sensitive information, including user credentials and student records, relying on established authentication solutions can reduce security risks and development complexity. If no existing identity authenticator is available, the system may implement local account management based on the school's records with secure password hashing and verification. Role-based access

control will be used to ensure that users can only access information that their roles permit.

2. *Web Server*

   An established webserver will host the site on an external hosting provider, allowing parents and teachers to reach the system from home.

3. *Web Framework*

   A web framework will be used to facilitate the development and maintenance of the application. Using an established web framework reduces the need to develop common functionalities from scratch and promotes easier consistency and maintainability.

4. *Document generation library*

   Existing document processing libraries will be used to support the mapping of data into official templates for printing, and provide the functionality for working with document templates and common file formats.

5. *DBMS*

   MySQL will be used as the relational database management system for storing and managing student records, announcements, tickets, messages, document requests and related data. It was chosen because of the team's familiarity with the technology and its widespread reliability and compatibility.

d. Usability Design

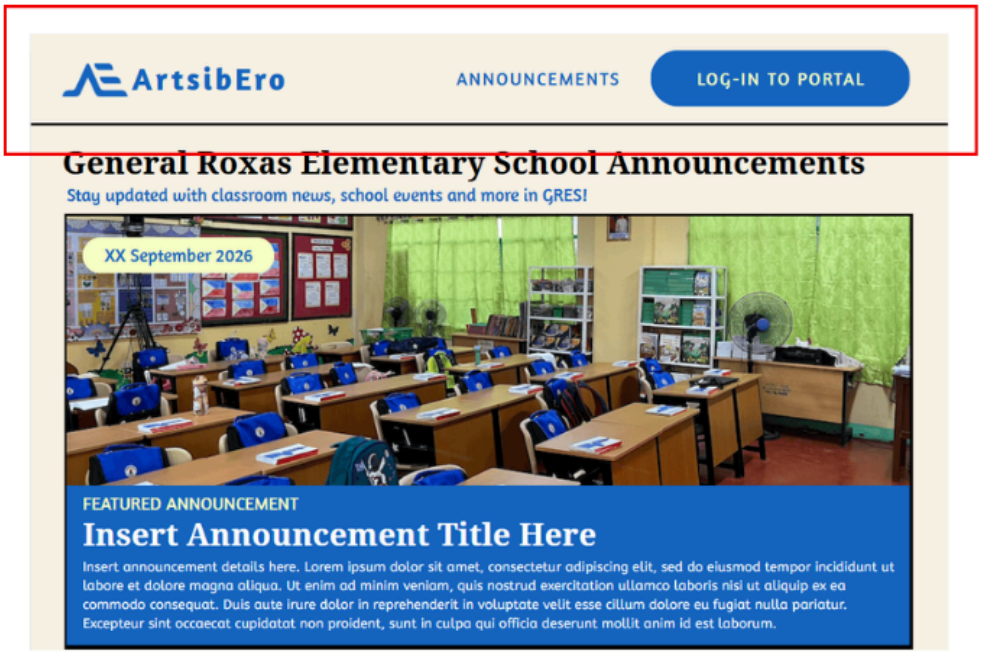
The ArtsibEro system generates several formal printed forms and periodic reports, which are essential to the user interface and overall user experience. Specifically, the system automatically maps student data and computed grades into official templates for printing. These output forms include the SF9 (Report Card), SF10 (Permanent Academic Record), Certificate of Enrollment, and more. Additionally, class records are formatted and exported into standard CSV files.

To ensure high usability, the interface design addresses functional issues by placing the user in control through straightforward and role-based workspaces for guardians, teachers, and admins. The system reduces the user’s memory load by using persistent

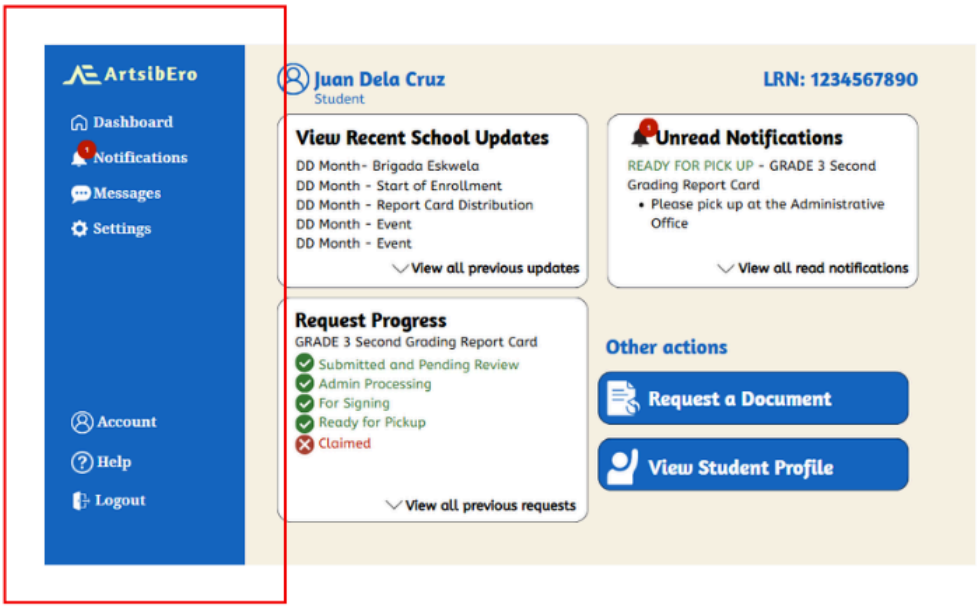
navigation sidebars and auto-mapping capabilities which also prevent manual errors. Task analysis models directly informed the display content to ensure that users are only presented with information relevant to their tasks (for example, teachers see attendance logs and grade sheets, while admins see document request queues and system logs).

i. **Interface Design (Menus and Links)**

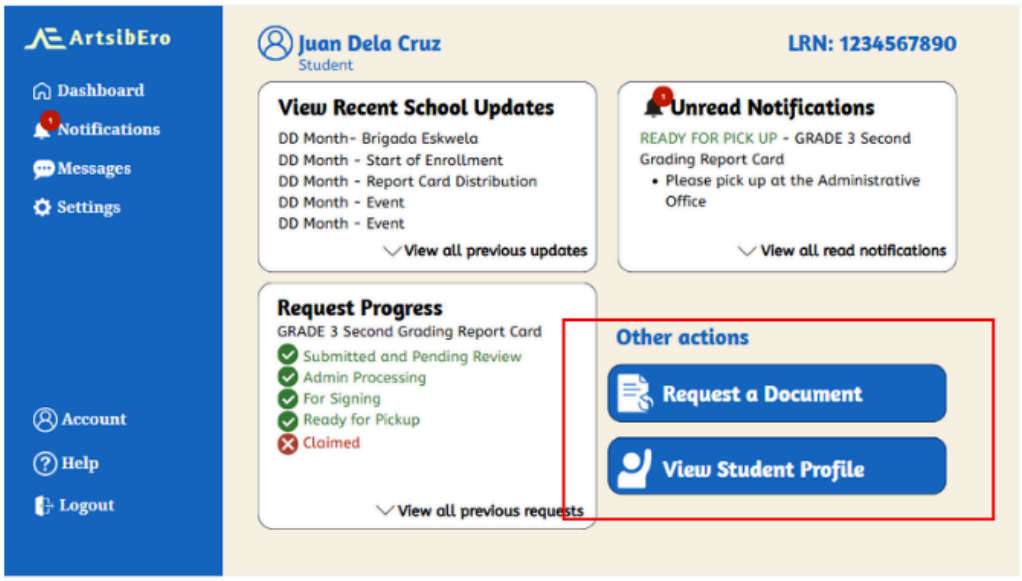
- Global Navigation - The public landing page uses a top navigation bar for announcements and portal login.



- Sidebar Navigation - Authenticated and permitted users (parents, teachers and admins) have a fixed left-hand sidebar containing primary buttons like the dashboard, notifications, and settings.



- Contextual links - Task-specific buttons like “Request a Document” are embedded within the main content area to guide users through sub-tasks.



ii. Aesthetic Design

The dashboard uses a modular card layout that separates different pieces of data into clean, rectangular containers with thin borders to make it easy to digest. The color scheme leans on a primary blue to highlight headers

and active buttons against a clear white and warm beige background. The blue and white were specifically chosen because they are General Roxas Elementary School's official colors, which helps make the system feel familiar to the community. Meanwhile, adding a warm neutral color like beige makes the website feel more inviting and comfortable to read for long periods. Standard sans-serif fonts were also used for readability and critical information was in bold to show a clear visual hierarchy.

iii. Navigation Design

Navigation design for ArtsibEro establishes how users move between functional modules and content objects. The system's navigation mechanics prioritize high-frequency tasks (such as document requests for parents and grades encoding for teachers) so they are reachable within one to two clocks from any dashboard. Links and menus are consistently fixed through a persistent sidebar to minimize disorientation and to ensure that navigation errors redirect back to the primary space.

Walkthrough of Typical Usage Scenarios

Scenario 1: Parent Requesting an Official Document (SF9)

1. Parent logs into portal using the linked email. (1 navigation click)
2. From the persistent sidebar, "Request a Document" link is clicked. (1 navigation click)
3. Selects the document type (SF9) and purpose (2 form-selection clicks)
4. Types contact details and authorized receiver name (around 25 keystrokes)
5. Clicks the "Submit" button. (1 navigation click)

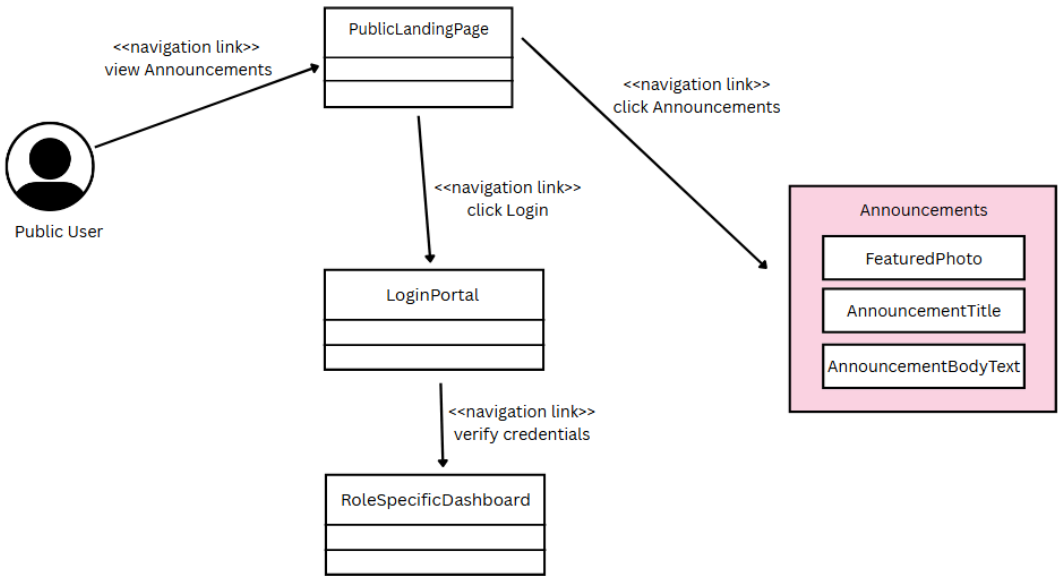
- User-Interface Navigation = 3 clicks
- Clerical Data Entry = 3 clicks + ~25 keystrokes = 28 actions
- Navigation accounts for ~9.7% of total actions, while clerical data entry accounts for 90.3%. This shows that the interface navigation is minimal

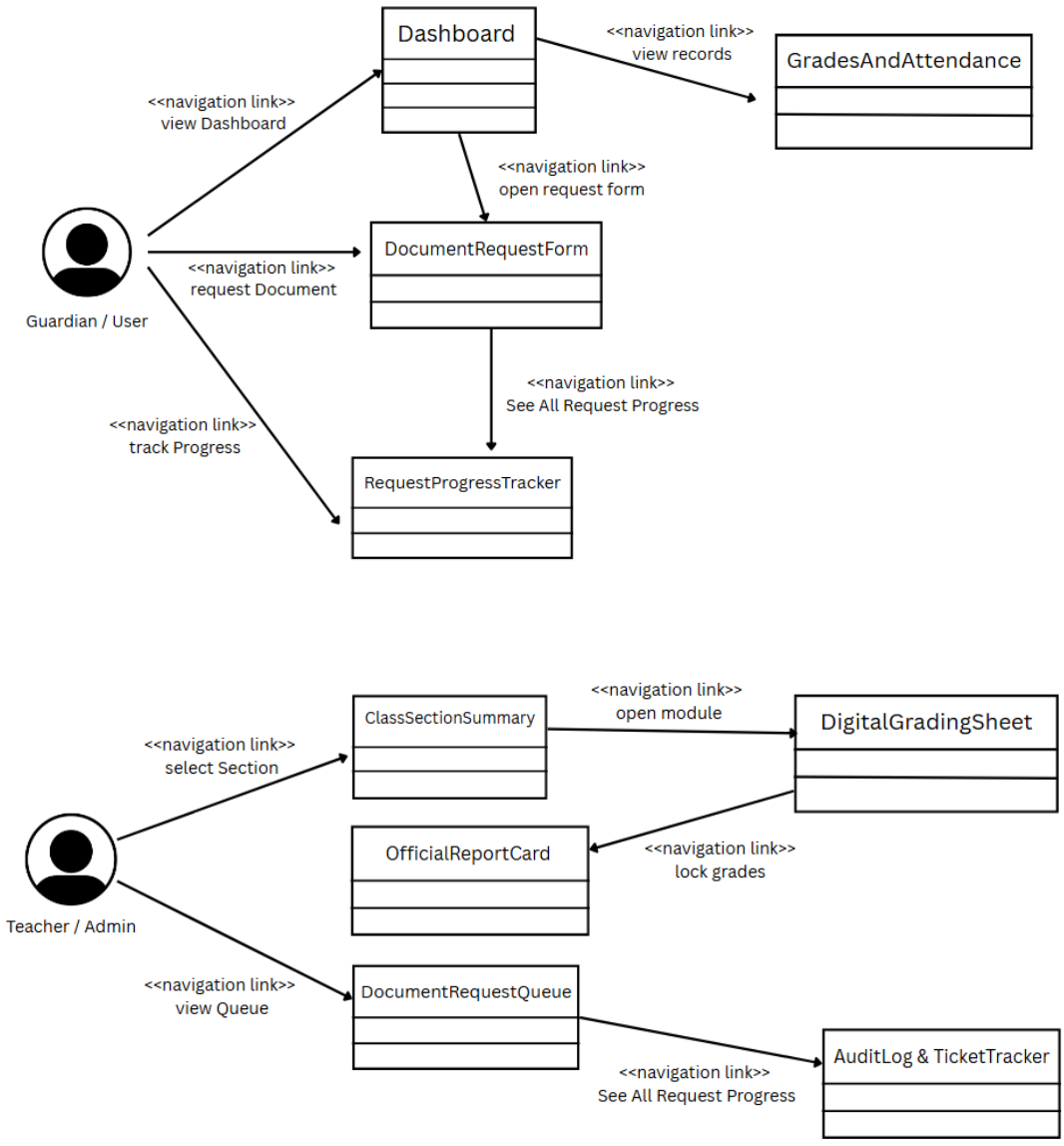
Scenario 2: Teacher Encoding Written Work Grades

1. Teacher logs into the employee portal using the linked email. (1 navigation click)
2. Selects the assigned class section from the dashboard (1 navigation click)
3. Opens the digital grading sheet module (1 navigation click)
4. Inputs raw scores for written works (~100 keystrokes)
5. Saves updated grades (1 navigation click)

- User-Interface Navigation = 4 clicks
- Clerical Data Entry = ~100 keystrokes
- Navigation accounts for ~3.8% of total actions, while clerical data entry accounts for 96.2%. This shows that the interface navigation is very minimal

Navigation Semantic Units





References:

IMI. (2020). *Access Control Types and Models*. Identity Management Institute.

https://identitymanagementinstitute.org/access-control-types-and-models/